Протокол виконання лабораторної роботи №3

з криптографії

Виконали студенти 3 курсу, НН ФТІ, ФБ-43

Закусило Дмитрій, Топчій Микита

Варіант 6

КРИПТОГРАФІЯ

КОМП’ЮТЕРНИЙ ПРАКТИКУМ №3

Криптоаналіз афінної біграмної підстановки

Мета роботи

Набуття навичок частотного аналізу на прикладі розкриття моноалфавітної підстановки; опанування прийомами роботи в модулярній арифметиці.

Порядок виконання роботи

0. Уважно прочитати методичні вказівки до виконання комп’ютерного практикуму.

1. Реалізувати підпрограми із необхідними математичними операціями: обчисленням оберненого елементу за модулем із використанням розширеного алгоритму Евкліда, розв’язуванням лінійних порівнянь. При розв’язуванні порівнянь потрібно коректно обробляти випадок із декількома розв’язками, повертаючи їх усі.

Пояснення коду програми Task1.py

extended_gcd(a, b) - Обчислює одночасно два ланцюги - остачі та їхні коефіцієнти при a і b:

Послідовність остач: r0 = a, r1 = b, кожна наступна r(i+1)= r(i-1) – q(i)*r(i)

Послідовності коефіцієнтів: кожна остача представлена як r(i) = u(i)*a + v(i)*b, і коефіцієнти оновлюються за тією ж формулою, що й остачі u(i+1)= u(i-1) - q(i)*u(i), аналогічно v.

Тут a,b = поточна пара остач r(i), r(i+1). u0, v0 = коефіцієнти першої остачі пари. u1, v1 = коефіцієнти другої остачі пари.

Кожна ітерація робить крок алгоритму Евкліда відразу для всіх трьох послідовностей: обчислює q, нову остачу a - q*b та її коефіцієнти u0 - q*u1, v0 - q*v1, потім зсуває пари змінних: старий u1 стає "першим", новий - "другим".

Коли b = 0, у «a» лежить остання ненульова остача - це d = gcd. Повернута трійка (a, u0, v0) задовольняє тотожність Безу: a*u0 + b*v0 = d.

mod_inverse(a, n) – шукає обернене до «a» за модулем «n», це можливо за умови gcd(a, n) = 1, результат отримується при виконанні розширенного алгоритму Евкліда: при gcd(a, n) = 1 тотожність Безу має вигляд a*u + n*v = 1, зведення за модулем n дає a*u ≡ 1 (mod n), отже u і є оберненим.

solve_congruence(a, b, n) - розв’язує ax≡b (mod n), логіка випливає з одного спостереження: розглядаються випадки:

1) d не ділить b – розв’язків нема.

2) d ділить b, тоді, розв’язуємо скорочене рівняння a1*x≡b1(mod n1). x0=b1*(a1^-1) (mod n1), в коді це «(u * (b // d)) % n1», адже u=(a1^-1), (після ділення на d тотожності Безу, u задовольняє отриману тотожність). Далі до знайденого базового розв'язка додаємо k*n1 для (k=0,...d-1), отримуємо d розв'язків, які знаходитимуться в межах n. На виході маємо всі розв'язки початкового рівняння.

Демонстрація роботи програми

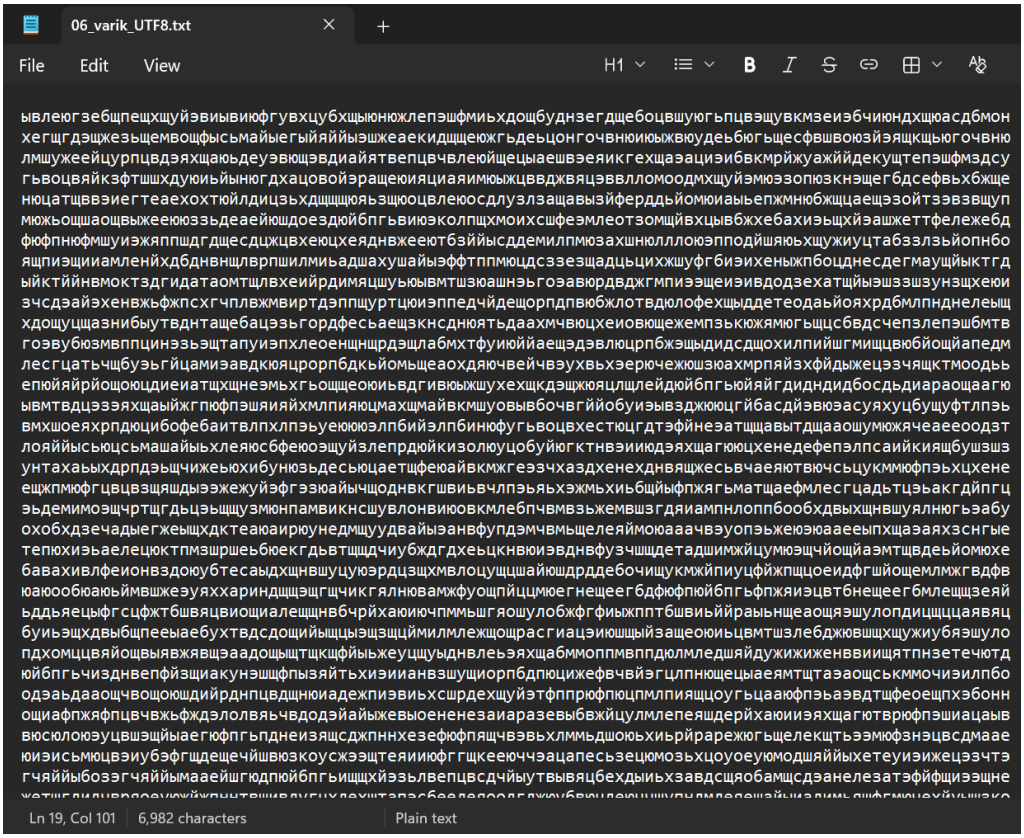
```
PS C:\Users\user\Desktop\Cryptography\Lab3> python -u "c:\Users\user\Desktop\Cryptography\Lab3\Task1.py"
641
1
None
[21]
[2, 6]
[]
[2, 6]
```

Аналіз отриманих результатів:

Програма працює, розв'язує тестові задачі правильно, коректно виводить результати, зокрема і для кількох розв'язків при розв'язанні рівняння.

2. За допомогою програми обчислення частот біграм, яка написана в ході виконання комп'ютерного практикуму №1, знайти 5 найчастіших біграм запропонованого шифртексту (за варіантом).

Вигляд шифротексту до шифрування:



Пояснення коду програми Task2.py

Програма, використовуючи функцію з лаби1, виводить топ-5 неперекривних біграм(саме з такими працює шифр який ми зламуємо)з наведеням кількості збігів.

Демонстрація роботи програми

```
PS C:\Users\user\Desktop\Cryptography\Lab3> python -u "c:\Users\user\Desktop\Cryptography\Lab3\Task2.py"
ще: 46
хе: 44
чв: 42
ле: 40
цв: 38
```

Аналіз отриманих результатів

Окремо тут варто проговорити про пункт з методичних вказівок, який стосується завдання безпосередньо:

П’ятьма найчастішими біграмами російської мови (в порядку спадання частот) є біграми «ст», «но», «то», «на», «ен». Перевірте ці відомості за допомогою програми підрахунку частот біграм з комп’ютерного практикуму №1.

Запустивши програму з лаби 1 на прикладі тексту без пробілів(«Война и мир», 3Мб тексту), замінивши топ-5 на топ-15, отримали такий результат:

то: count=12525, freq=0.01682
ст: count=9831, freq=0.01320
на: count=9021, freq=0.01212
ов: count=8584, freq=0.01153
ал: count=8198, freq=0.01101
он: count=8084, freq=0.01086
не: count=8010, freq=0.01076
го: count=7976, freq=0.01071
но: count=7858, freq=0.01055
ко: count=7733, freq=0.01039
ос: count=7569, freq=0.01017
по: count=7510, freq=0.01009
ен: count=7320, freq=0.00983
ра: count=7025, freq=0.00944
от: count=6860, freq=0.00921

З виводу видно, що результат збігається лише частково з тим, який очікується в методичних вказівках. Тим не менше, ми вважаємо реалізацію програми пошуку неперекривних біграм коректною, як і результат який вона дає:

усі п'ять біграм входять у топ-13, топ-3 збігається, частоти топ-3 близькі до класичних. Розбіжності в середній частині списку пояснюються жанровою специфікою обраного тексту, наприклад підвищена частота «ов», «ал» - відмінкові та дієслівні закінчення військового наративу тощо.

Оскільки атака, реалізована в подальших пунктах, перебирає всі варіанти співставлення частих біграм мови та частих біграм шифртексту, а не покладається на точне співпадіння порядку рангів, достатньо, щоб правильні відповідності біграм входили в обидві п'ятірки. Тому для реалізації наступних пунктів лаби використано список з методичних вказівок.

3. Перебрати можливі варіанти співставлення частих біграм мови та частих біграм шифртексту (розглядаючи пари біграм із п’яти найчастіших). Для кожного співставлення знайти можливі кандидати на ключ (,) ab шляхом розв’язання системи (1).

Пояснення коду програми Task3.py

Програма перебирає всі можливі зіставлення частих біграм мови та частих біграм даного шифротексту, для кожного розв'язує систему (1) з методички і формує множину кандидатів на ключ (a, b).

Після читання шифротексту, контролю алфавіту та знаходження п'яти найчастіших біграм шифротексту запускається чотири вкладені цикли, які перебирають усі гіпотези про відповідність біграм:

Два зовнішні цикли генерують усі впорядковані пари біграм мови X*, X** з п'ятірки методички (5*4 = 20 пар, однакові біграми пропускаються через continue, бо при них різниця дельтаX нульова і порівняння вироджене), а два внутрішні - такі ж 20 пар біграм шифртексту Y*, Y**, разом 400 гіпотез. Кожна гіпотеза означає: "якщо X* шифрувалась у Y*, а X** - у Y**, то ключ задовольняє систему $Y \equiv aX + b \pmod{961}$"; поелементне віднімання рівнянь системи знищує b і зводить задачу до лінійного порівняння $\text{дельтаY} \equiv a * \text{дельтаX} \pmod{961}$, яке розв'язує функція solve_congruence з Task1, повертаючи всі розв'язки. Цикл «for a» при цьому обробляє всі випадки: порожній список означає суперечливу гіпотезу (тіло циклу не виконується), кілька розв'язків - кілька окремих кандидатів. Кожен розв'язок a перевіряється на оборотність (gcd(a, 961) = 1 - умова існування ключа), а відповідне b обчислюється підстановкою в перше рівняння системи: $b = (Y^* - a * X^*) \bmod 961$. Результати накопичуються в множину set, яка автоматично прибирає дублікати, що виникають від різних гіпотез, - серед них є справжній ключ, якщо правильні відповідності біграм входять в обидві п'ятірки.

Демонстрація роботи програми

Наведено неповний вивід програми, а лише частину кандидатів, повний вивід скопійовано у файл task3full.txt (350 рядків виводу)

```
PS C:\Users\user\Desktop\Cryptography\Lab3> python -u
"c:\Users\user\Desktop\Cryptography\Lab3\Task3.py"
cipher top5: ['ще', 'хе', 'чв', 'ле', 'цв']
candidates: 348
a=2, b=473
a=2, b=501
a=2, b=811
a=6, b=135
a=7, b=620
a=7, b=744
.....
a=948, b=81
a=952, b=689
a=954, b=751
a=954, b=816
a=955, b=303
a=959, b=529
```

Аналіз отриманих результатів:

маємо 348 із 400 можливих пар: частину відкинули через суперечливість лінійного порівняння, непроходження фільтру оборотності, або дублювання вже відібраної пари. Частина гіпотез дала по кілька розв’язків – кожен з них вважаємо кандидатами: різні гіпотези зійшлися на одному значенні a. або b

Справжній ключ гарантовано входить у множину за умови, що правильні відповідності частих біграм мови та шифртексту знаходяться в межах обох п'ятірок.

4. Для кожного кандидата на ключ дешифрувати шифртекст. Якщо шифртекст не є змістовним текстом російською мовою, відкинути цього кандидата.

5. Повторювати дії 3-4 доти, доки дешифрований текст не буде змістовним.

Пояснення коду програми Task4.py

Розшифрування decrypt. Для кожного кандидата (a, b) обчислюється обернений елемент a^-1 за модулем 961 через mod_inverse з Task1 (один раз на ключ), далі текст проходом по неперекривних біграмах перетворюється за формулою методички X = (a^-1)*(Y − b) mod 961: кожна шифр-біграма переводиться в число, застосовується обернене перетворення, результат розпаковується назад у дві букви через ділення націлено та остачу.

Статистика на яку ми орієнтуємося build_reference. Зі справжнього тексту лаби 1 ("Война и мир", ≈3 МБ) підраховуються частоти всіх перекривних біграм, статистику яких правильне розшифрування має відновлювати.

Критерій змістовності score. Лінійний частотний критерій: для розшифрованого тексту беруться всі перекривні біграми, для кожної береться її еталонна частота (0, якщо біграма в відсутня в еталоні), і сума нормалізується на кількість біграм - виходить середній еталонний вклад однієї біграми.

Загальний прохід. Кандидати на ключ генеруються так само, як у Task3. Для кожного з кандидатів виконується розшифрування та оцінка, після чого список (a, b, score) сортується за score, найкращий кандидат розшифровується фінально, зберігається у decrypted.txt, і його частина виводиться в консоль.

Демонстрація роботи програми

```
PS C:\Users\user\Desktop\Cryptography\Lab3> python -u
"c:\Users\user\Desktop\Cryptography\Lab3\Task4.py"
reference score: 0.004921
tested: 348 candidates
a=441, b=310, score=0.004571
a=844, b=434, score=0.002240
a=69, b=713, score=0.002215
a=751, b=775, score=0.002181
```

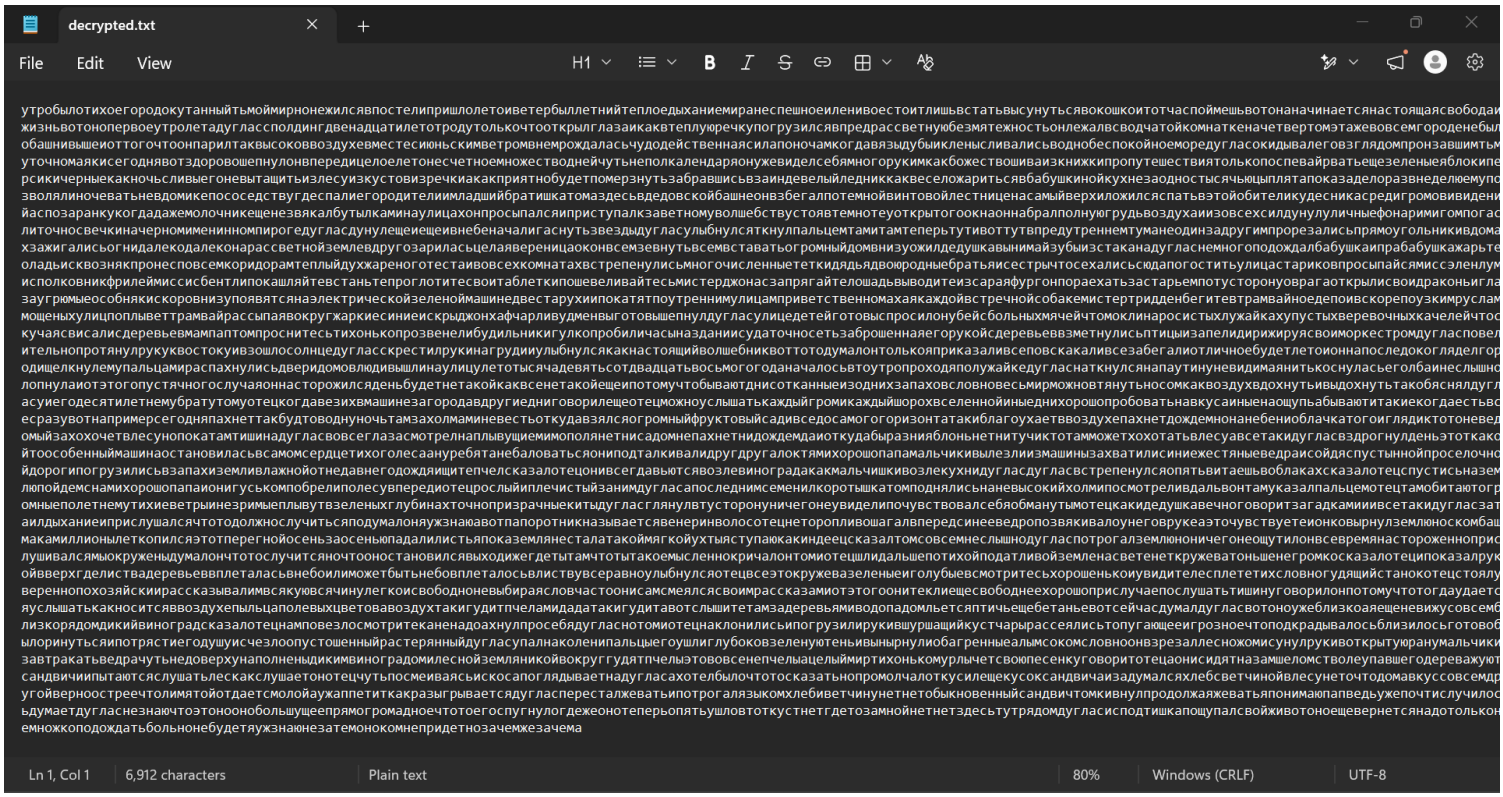
a=7, b=620, score=0.002180
a=534, b=930, score=0.002173
a=410, b=744, score=0.002168
a=720, b=248, score=0.002164
a=472, b=837, score=0.002161
a=38, b=186, score=0.002157

best key: a=441, b=310
утробылотихоегородокутанныйтьмоймирнонежилсявпостелипришлолетоиветербыллетнийтеплоедыханиемиранеспешноеиленивоестоитлишьвстатьвысунутьсявокошкоитотчаспоймешьвотоначинаетсянастоящаясвободаижизньвотон

Аналіз отриманих результатів:

Еталонний текст дає score = 0.004921. Розшифровка правильним ключем дає 0.004571 - значення того ж порядку, невелике відхилення пояснюється жанровою відмінністю тексту варіанта від еталонного тексту. Неправильні кандидати зібралися в діапазоні 0.002, що відповідає розшифруванню випадковим ключем.

Правильний ключ a = 441, b = 310 має двократний відрив від найкращого неправильного кандидата (0.004571 проти 0.002240) та явно відокремлений від решти групи. Розшифрований текст змістовний, розшифрували правильно.



Контрольні питання

1. Які статистичні властивості шифртексту були використані під час формування кандидатів на ключ?

Використано той факт, що афінна біграмна підстановка зберігає частоти неперекривних бігам: найчастіші біграми шифртексту (ще, хе, чв, ле, цв) розглядалися як ймовірні шифрообрази найчастіших біграм мови (ст, но, то, на, ен). Це дозволило будувати гіпотези про відповідність біграм і зв'язувати їх із системою рівнянь на ключ.

2. Чому не кожне зіставлення частих біграм відкритого тексту та шифртексту приводить до допустимого ключа?

Бо гіпотеза про відповідність біграм може бути хибною - тоді система $Y^* \equiv aX^* + b \pmod{961}$ несумісна: лінійне порівняння дельтаY ≡ a*дельтаX (mod 961) не має розв'язків, якщо дельтаY не ділиться на d = gcd(дельтаX, 961). Крім того, розв'язок «a» може не пройти умову оборотності gcd(a, 961) = 1, без якої перетворення не є бієкцією і розшифрування неможливе.

3. Чи виникали під час криптоаналізу випадки, коли одне зіставлення біграм давало декілька кандидатів на ключ? Як такі кандидати оброблялися?

Так, виникали: коли різні гіпотези могли сходиться на одному a з різними b – коли різні гіпотези повертали зійшлися на одному a при різних b, також може бути й навпаки: одне зіставлення давало до d різних значень a. Усі розв'язки поверталися списком функцією solve_congruence і кожен окремо проходив фільтр оборотності та включався до множини кандидатів - жоден потенційний ключ не втрачався.

4. За якими ознаками серед отриманих кандидатів було визначено правильний ключ?

За різким відривом критерію змістовності розшифрованого тексту: правильний ключ (a=441, b=310) дав score = 0.004571 при найкращому неправильному 0.002240 - двократний відрив, тоді як решта кандидатів зібралася бля 0.0022. Розшифровка правильним ключем сильно ближче до score еталонного тексту за порядком і є зв'язною російською мовою.

5. Який критерій автоматичного розпізнавання осмисленого тексту було використано? Чому було обрано саме його?

Критерій частих перекривних біграм: для розшифрованого тексту обчислюється середній еталонний вклад біграми. Обрано його тому, що частотний профіль біграм російської мови різко нерівномірний, а неправильний ключ руйнує відповідність частот, тому середній вклад розділяє мову та шум.

6. Як поводився обраний критерій для правильного відкритого тексту та для результатів розшифрування неправильними ключами?

Для правильного тексту - високе значення 0.004571, близьке до еталонного рівня мови (0.004921 для "Война и мир"). Для всіх неправильних ключів - низькі значення в вузькому діапазоні 0.002, без перетинання з групою правильного ключа: жоден хибний кандидат навіть наблизився до порогу осмисленості, запас від топ-2 удвічі.

Висновки

У ході роботи ми реалізували математичний апарат модулярної арифметики - розширений алгоритм Евкліда, обчислення оберненого елемента за модулем та розв'язання лінійних порівнянь з коректною обробкою всіх випадків. За їх допомогою виконали повний криптоаналіз шифру афінної біграмної підстановки на основі частотного аналізу: побудували множину кандидатів на ключ шляхом перебору співставлень частих біграм мови та шифротексту, розробили автоматичний розпізнавач російської мови на основі частотного критерію перекривних біграм та відібрали з множини кандидатів справжній ключ, який дав осмислений текст.